02 — Engineering & Operations Lessons

Cập nhật 2026-11-07, branch *`hmm-2nd-idea`*. - **Phạm vi:** lập trình, môi trường, Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa, bài học. Không chứa kết quả khoa học (→ `01`) hay kế hoạch (→ `03`). - Bảng tài khoản / dataset: `ENVIRONMENT_SUMMARY.md`. - Bản đầy đủ trước đó: `history/02_ENGINEERING_OPS_NOTES_through_2026-11-06.md` (ý tưởng 2a HSI, AUTOLOOP, engine GPU) và `history/02_ENGINEERING_OPS_NOTES_full_through_2026-11-03.md` (ý tưởng 1).

1. Tài nguyên và quy tắc dùng (theo `HMM_Navigation_Yeu_Cau_Nghien_Cuu_Cap_Nhat.docx`, đã kiểm chứng)

Tài nguyên	Thực tế	Quy tắc
Phiên L4 Kaggle	4× NVIDIA L4 (23 GB), 48 vCPU, 188 GB RAM; 2 phiên / tài khoản; 30 GPU-giờ / tuần / tài khoản	Mọi đợt OWF chạy L4, `--par 24` (6 job / GPU; phần HSI dùng CPU của cùng phiên)
Phiên CPU	4 vCPU; 5 phiên / tài khoản; không tốn hạn mức GPU	Sinh kho seed (`hsi_fresh.py --keep`, ~1.5 h / kernel)
Thứ tự lấp	—	Lấp hết slot của một tài khoản rồi mới sang tài khoản kế (`automation/active_accounts.txt`; monitor tự làm)
Kẹt hàng đợi	—	Không có output 8 phút (L4/CPU) / 10 phút (T4): xoá + đẩy lại ≤ 5 lần, soát lỗi code, rồi 2×T4 (monitor tự làm)
API Kaggle	Giới hạn theo IP	**Một** `central_monitor.py` cho mọi đợt. Muốn thêm kernel: dừng rồi chạy lại với cả danh sách (state giữ nguyên)
Token	Người dùng đưa trong chat	Chỉ lưu `~/.kaggle/tok_<username>` (chmod 600). Không bao giờ vào repo / dataset / báo cáo. Kiểm `git grep KGAT_` trước khi kết thúc

2. Quy trình chuẩn của một đợt (đã dùng cho mọi đợt Entry 19–27)

1. **Thử cục bộ trước** (vài phút): `owf.py <cache> <dump> <ds> <desc> <seed> --qx 0.02 --scales 1 [c■ m■i]` trên 2–3 run thiết kế (CMU s60, Nordland s58, old town s58 CosPlace; dump cục bộ `scratchpad/data/dump`).
2. **Nhóm job:** `automation/owf_wave.py <groups.txt> --set inc:<t■p> [--pool rXX] --tag <T> --owfextra "<c■ owf.py>" [--sota] --kernels N --par 24`. - Tập: `setA` (phát triển, 405 job), `fresh<a>-<b>` (nhóm seed mới), `holes`, `natural`, `nclt`. - `--sota` thêm `sota_job.py` (TAO'26, Xu'21, Claxton'24, top-1, oracle) trên cùng run.
3. **Dựng kernel:** `VPR_CODE_DS=dive-vpr-owf automation/fanout.py <work> <prefix> <groups.txt> --groups --build-only --l4 --par 24 --nocopy --ds <dataset ngu■n>`. Bước này tự đồng bộ code dataset của mọi tài khoản và chờ "ready". - Tập A cần `nhatlong1103/hmm-navcache-v2, hmm-e37-caches, nclt-vpr-dump, oxford-robotcar-vpr-dump, stlucia-vpr-dump`. - Nhóm seed mới cần `namhunhh/hmm-navcache-rXX` + Oxford + St Lucia dump.
4. **Monitor:** `VPR_CODE_DS=dive-vpr-owf nohup automation/central_monitor.py <state.json> --gap 15 --run-poll 180 --nocopy --pull-pattern '^(results\.tar|.*\.log)$' <kernel dirs...> >> monitor.log &`.
5. **Gom:** copy `fr_*_<tag>_b1.npz` và `hsifr_*_<tag>xms09_16.npz` từ `<kernel>_out/results` sang một thư mục làm việc, rồi: - `owf_union.py <fr> <h> <navcache> <out.json> --tag <tag>xms09_16 --ftag _<tag> --variants "H|m(1e-5,16)|m(1e-8,2)|t(3e-4,0.5);H"`; thêm `OWF_VETO=0.5` cho điểm vận hành an toàn; - so sánh: `hcal_board.py <merged.json> --ref <R> --variants ...` (bootstrap CI theo run).
6. **Kiểm mã đúng phiên bản:** mỗi JSON `owf_*.json` của luật cuối phải có `lu.mode = "hsq2"`. `owf.py` bỏ qua giá trị lạ của `--lu`, nên kernel chạy mã cũ vẫn ra kết quả nhưng sai luật.

Chi phí đo được (L4): - ngân hàng 10 bộ lọc 2.7–3.4 ms/frame; - luật cuối (2 lượt) ~6 ms/frame; - tập A (405 run) ~30 phút trên 8 phiên; - một nhóm seed mới (144 run, có SOTA) ~25 phút trên 9 phiên; - sinh kho seed 288 cache ~1.5 giờ trên 20–24 kernel CPU.

3. Tổ chức thử nghiệm

- **Đăng ký trước mọi quyết định.** Tiêu chí + quy tắc viết vào `RESEARCH_LOG.md`, commit + push TRƯỚC khi kernel kiểm tra chạy. Giờ ghi theo `date -u`.
- **Script đánh giá phải in đúng tên luật đã đăng ký.** Entry 23: script sao từ Entry 21 in nhầm quyết định. Quyết định được tính lại tay từ luật đã commit.
- **Phân chia dữ liệu:**
- thiết kế: vài run, cục bộ;
- phát triển: tập A, 399 run, đọc tuỳ ý;
- kiểm tra: một nhóm 3 seed CHƯA dùng cho mỗi quyết định.
- Sổ dùng nhóm seed: `03_HANDOFF.md` §5. Kho còn trống: `namhunhh/hmm-navcache-r73` (seed 73–78).
- **Sinh kho seed một lần, publish công khai, mount nhiều lần.** Kernel sinh cache là private, nên phải kéo về rồi publish dataset (không token). `fetch_done.py <state> <dest> '^frXX-' --pattern '^navcache_.*\.npz$'`, rồi kiểm `np.load` từng file, rồi `publish_dataset.py <dir> hmm-navcache-rXX "<title>" "<subtitle>" --owner namhunhh`.
- **HSI-VN của kho mới nằm trong `results.tar`** của từng kernel `frXX-*`, không phải file rời. Kéo `--file-pattern '^results\.tar$'` rồi giải nén `hsi_*.json`.
- **Một cờ cho mỗi thay đổi**, mặc định = hành vi cũ (`--pin`, `--lu`, `OWF_VETO*`). Ablation và tái lập vì thế dễ.
- **Kiểm thử tính nhân quả** cho mọi ước lượng "từ dữ liệu của chính phiên": đổi tương lai, quá khứ phải không đổi (`analysis/tests/test_hq_causality.py`). Bài kiểm này bắt được hai rò rỉ mà mắt không thấy (Entry 27).

4. Lỗi đã gặp và cách sửa (phiên 2026-11-05 → 07)

Lỗi	Triệu chứng	Sửa / phòng
Rò rỉ tương lai trong hiệu chỉnh HQ	LR ở frame t phụ thuộc frame sau (sàn KDE lấy biên từ khối 50 frame; stride thưa mẫu theo tổng số frame tự tin của phiên)	`--lu hsq2` + kiểm thử nhân quả; chạy lại mọi kết quả (Entry 27)
Script đánh giá sao chép	In dòng DECISION của luật khác	Tính lại từ luật đã commit; ghi chú trong log
Đọc nhầm phân vị	Ghi trung vị z bí danh 3.27 (thật ra phân vị 25 %; trung vị 4.09)	In thống kê có nhãn rõ; đã đính chính
Đĩa scratchpad đầy (~40 GB)	`fetch_done` ghi file cụt mà vẫn đánh dấu "đã kéo"	Xoá `runs/*/<kernel>_out` sau khi trích; kiểm `np.load` từng file; kéo lại
Container khởi động lại (3 lần)	Monitor chết	Chạy lại monitor từ state file. Kernel đã xong khi monitor chết: `kaggle kernels status` rồi kéo tay
Code dataset chưa sẵn sàng	Kernel chạy mã cũ, cờ mới bị bỏ qua	`fanout` chờ "ready"; kiểm `lu.mode` trong JSON đầu ra
L4 cần chấp nhận luật cuộc thi	"You must accept this competition's rules"	Người dùng bấm chấp nhận tại trang `gemma-4-developer-agent`
Hết hạn mức GPU tuần	"Maximum weekly GPU quota of 30.00 hours"	Monitor tự chuyển tài khoản; hạn mức reset theo tuần
Tài khoản "chỉ T4" nhận kernel L4 nhưng chạy rất chậm	ablation > 1.5 giờ	Xoá kernel, dựng lại nhóm đó trên tài khoản L4
`kaggle kernels output` trên kernel output lớn	429 nhiều chục phút	`--pull-pattern` chọn lọc; mount output làm kernel source
f-string có `{...}` trong script ghi tài liệu	SyntaxError, commit rỗng	Dùng chuỗi thường + `.replace()`

5. Bài học

- Thứ tự hiệu quả: chẩn đoán từng frame / từng run thất bại → mệnh đề viết trước → thử cục bộ 3 run → đợt phát triển → đăng ký → nhóm seed mới. Hai phát hiện lớn của phiên (bí danh dịch chuyển, rò rỉ nhân quả) đều đến từ việc nhìn vào từng run cụ thể, không phải bảng gộp.
- Thử cục bộ trên đúng run thất bại tiết kiệm nhiều đợt GPU. hs / hsc / hsr / hsb / hsqsel đều bị loại ở bước này.
- Tài nguyên song song: trong khi GPU chạy kiểm tra, CPU sinh kho seed kế tiếp. Nhờ vậy quyết định không phải chờ dữ liệu.
- Mọi khẳng định "nhân quả", "không nhãn", "không chỉnh theo dataset" cần một kiểm thử hoặc một dòng mã chứng minh. Không đủ nếu chỉ dựa vào mô tả.

6. Lệnh của luật cuối (OWF-HWA-S5-NT-HQ, mã đã kiểm toán)

```
owf_hsi_job.py <cache> <dump> <ds> <desc> <seed> --tag T --modes xms0.9_16:HSI-VN --hsiframes \
    --qx 0.005,0.02 --scales 0.94,0.97,1.0,1.03,1.06 --engine torch --pin 0 --lu hsq2
owf_union.py <fr_dir> <hsifr_dir> <navcache_dir> out.json --tag Txms09_16 --ftag _T --variants "H|m(1e-5,16)|
OWF_VETO=0.5 owf_union.py ...          # ■i■m v■n hành an toàn
```